

Software Engineering II

Semester 2026-I

Workshop No. 4 — Full Software Testing, Deployment, and MVP Delivery

Eng. Liliana Marcela Olarte, M.Sc.

Eng. Carlos Andrés Sierra, M.Sc.

Lecturers

Computer Engineering Program

School of Engineering

Universidad Nacional de Colombia

Welcome to Workshop 4! This session focuses on delivering your first complete Minimum Viable Product (MVP) for your course project, including full software testing, deployment, and comprehensive documentation. The goal is to ensure your application is robust, validated, and ready for demonstration in a production-like environment with a **react-native** mobile frontend, **Spring Boot/SQLAlchemy** backends with proper ORMs, and complete **Swagger/OpenAPI** API documentation.

Scope and Objectives

- **Full Software Testing:** Implement comprehensive unit tests using **JUnit** (Java) and **pytest** (Python), integration tests for ORM layers, and acceptance tests for key user stories.
- **Deployment:** Containerize your application using **Docker** and orchestrate services with **Docker Compose**. Prepare deployment scripts and instructions.
- **CI/CD Pipeline:** Set up a **GitHub Actions** workflow for automated testing, building, and deployment of all services.
- **MVP Delivery:** Deliver a fully functional MVP integrating **react-native** mobile frontend, **Spring Boot/FastAPI** backends with ORM layers, relational database, and complete **Swagger/OpenAPI** API documentation.

Carlos Andrés Sierra, Computer Engineer, M.Sc. in Computer Engineering, Lecturer at Universidad Nacional de Colombia.

Any comment or concern regarding this workshop can be sent to Carlos A. Sierra at: *casier-rav@unal.edu.co*.

Methodology and Deliverables

1. Testing

- Provide unit tests for backend components using `JUnit` (Java) and `pytest` (Python).
- Implement integration tests to validate interactions between system modules.
- Develop acceptance tests for key user stories (e.g., using `Cucumber` or end-to-end testing).
- Include test results and coverage reports in your `README.md`.

2. Deployment

- Create Dockerfiles for all major components (backend, frontend, database).
- Provide a `docker-compose.yml` to orchestrate the application stack.
- Document deployment steps and environment configuration in your `README.md`.

3. CI/CD Pipeline

- Implement a CI/CD workflow using GitHub Actions or similar, including steps for running tests, building Docker images, and deploying containers.
- Include evidence of successful pipeline runs (e.g., screenshots, logs).

4. MVP Delivery

- Deliver a working MVP that covers the main functional requirements and user stories.
- Ensure the MVP integrates `react-native` mobile frontend, `Java/Python` backends with ORM layers, and relational database.
- Provide complete `Swagger/OpenAPI` API documentation with interactive `Swagger` UI for backend endpoints.
- Provide a brief demo (screenshots or video) showing the main features, API documentation, and deployment.

5. Documentation

- Include comprehensive `Swagger/OpenAPI` API documentation for all backend endpoints.
- Compile all deliverables into a single PDF.
- Organize your files in a folder named `Workshop-4` in your course project repository, with a `README.md` referencing each section, providing setup instructions, and linking to API documentation.

Project Requirements Checklist

- Comprehensive unit tests for **Java** (JUnit) and **Python** (pytest) backends.
- Integration tests validating backend services with ORM layers (**JPA/Hibernate** and **SQLAlchemy**).
- Acceptance tests for key user stories.
- Dockerfiles and `docker-compose.yml` for deployment.
- CI/CD pipeline for automated testing and deployment.
- Functional MVP with **react-native** mobile frontend, **Spring Boot/FastAPI** backends, and relational database.
- Complete **Swagger/OpenAPI** API documentation with Swagger UI.
- Organized and referenced documentation.

Recommended Technology Stack

This workshop recommends the following technology stack aligned with Workshop 3 deliverables:

- **Backend Testing:** JUnit for Java (**Spring Boot**), **pytest** for Python (**FastAPI/Flask**).
- **Integration Testing:** **Testcontainers** for database integration tests.
- **API Documentation:** **Swagger/Springdoc** for Java, **Swagger/OpenAPI** for Python.
- **Containerization:** **Docker**, **Docker Compose**, **Dockerfile** for all services.
- **CI/CD:** **GitHub Actions** for automated testing, building, and deployment.
- **Mobile Frontend:** **react-native** for cross-platform mobile application.
- **Database:** **PostgreSQL** or **MySQL** with ORM layers (**JPA/Hibernate**, **SQLAlchemy**).

Thursday, July 23rd, 2026, at 18:00. Late submissions will not be accepted.

Notes

- All documents must be in **English**.
- Ensure your tests cover the ORM layers (**JPA/Hibernate** for Java, **SQLAlchemy** for Python).
- Verify that **Swagger/OpenAPI** documentation is automatically generated and accessible via Swagger UI.
- Test your **Docker** containers locally before final submission.

- Cite any references (articles, tutorials, frameworks) that influenced your design choices.
- Focus on clarity, completeness, and professional presentation. This is your first full MVP delivery with production-like deployment.

Good luck! This workshop is your opportunity to demonstrate a robust, tested, and deployable MVP for your software engineering project.